

Lab 02: [numerical and process-based models]

Maitreyi Chakrapani (820264599)

2026-08-01

Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Part A: [Tracer decay and step size (numerical)]

Part A looks at the concentration of a tracer in the environment over time.

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# setting the parameters
C0, k, dt, t_end = 100.0, 0.05, 1.0, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
```

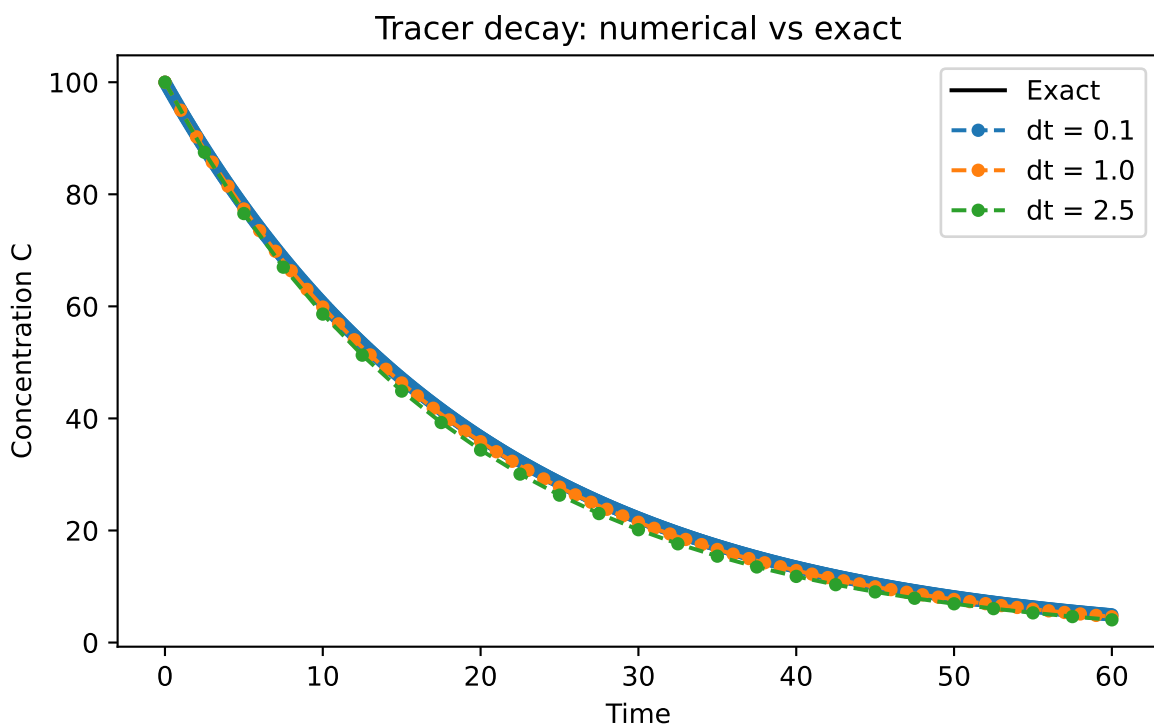
```

ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.1, 1.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

```



```

## Task 1: changing K
C0, k, dt, t_end = 100.0, 0.01, 1.0, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

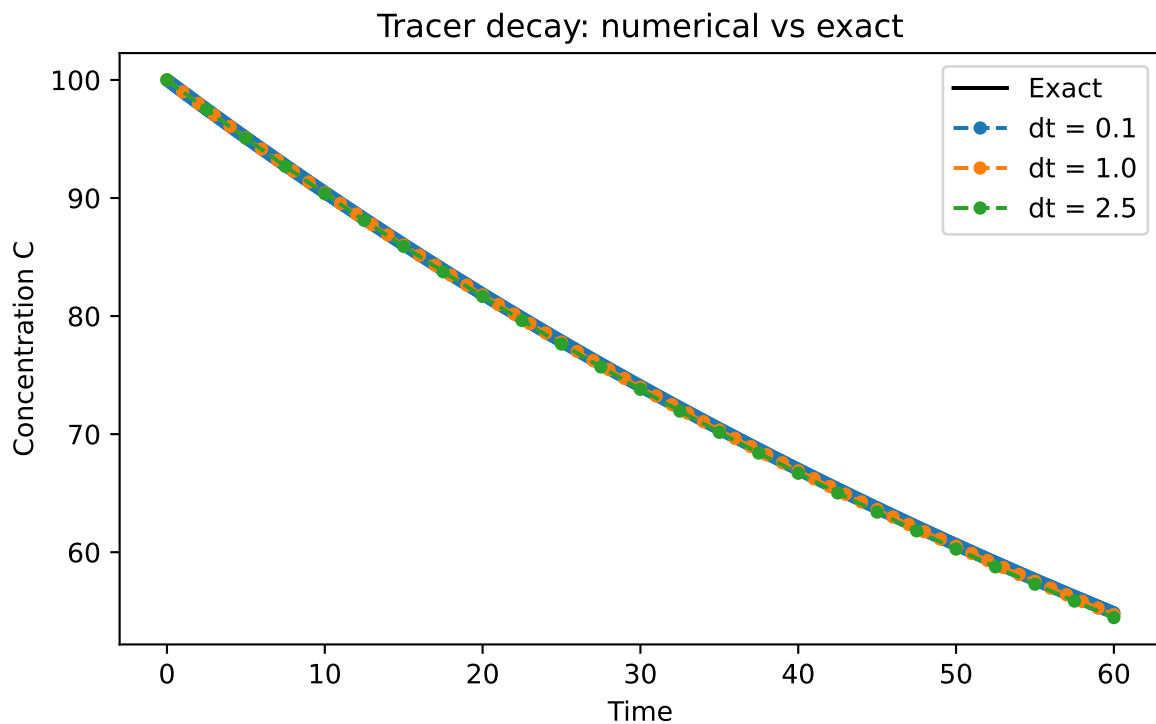
```

```

for dt in [0.1, 1.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

```



Answer to task A1: K controls the rate of decay. A smaller K value results in a slower decay, while a larger K value leads to a faster decay.

```

## Task 2: changing dt to 2.0 and 5.0
C0, k, dt, t_end = 100.0, 0.05, 2.0, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

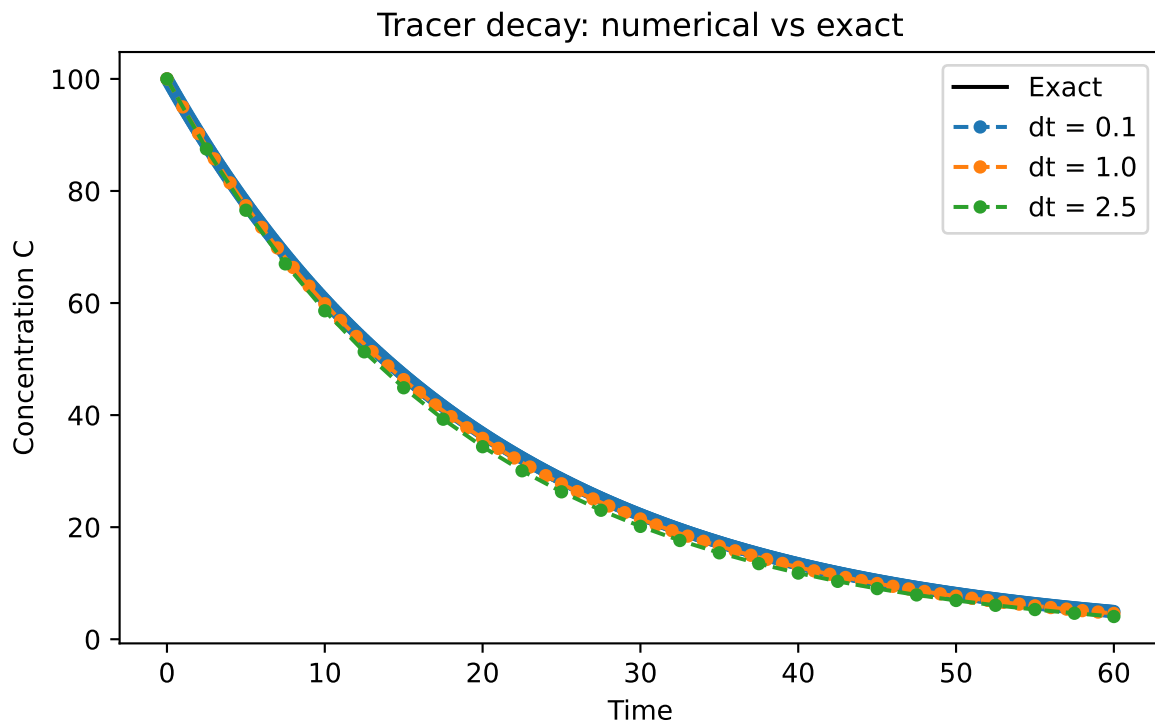
```

```

for dt in [0.1, 1.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

```



Answer to task A2: At value $dt = 2.0$, the curve is not as close to the Exact curve.

```

## Task 3: running code with 4 different dt values
C0, k, dt, t_end = 100.0, 1.0, 5.0, 6.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

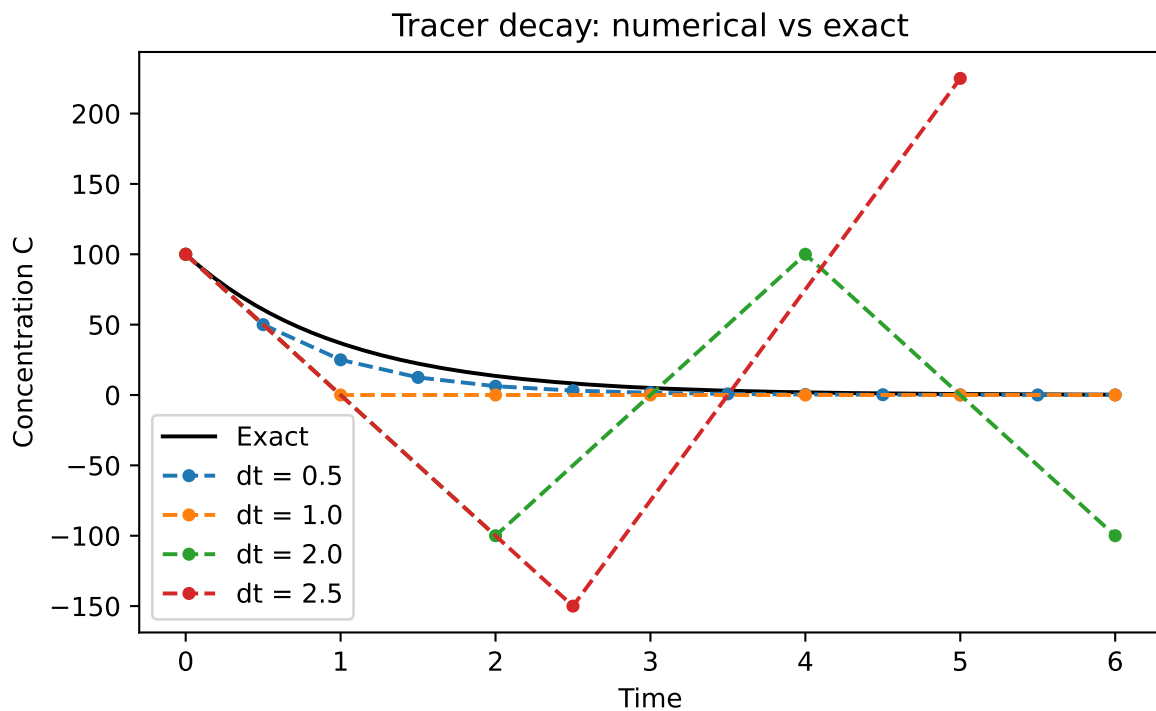
```

```

for dt in [0.5, 1.0, 2.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()

```



Answer to task A3: $dt = 1.0$. After time = 1, the curve reaches 0, and stays there.

Answer to task A4: If $k \cdot dt > 1$, the amount it tries to subtract is greater than C_{old} , which results in a negative concentration. Once it's negative, decay "in reverse" flips the sign again for the next time step, which is why the curve bounces back and forth.

Part B: [Reservoir box model (process-based)]

```

import numpy as np
import matplotlib.pyplot as plt

## Task 1: compute analytical steady state
# ---- Parameters ----
V = 1.0e6          # lake volume (m^3)
Q = 5.0e3          # river inflow (m^3 per day)
C_in = 10.0        # incoming concentration (mg/L)
k = 0.05           # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0           # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0            # clean lake

# ---- Storage ----
history = [C]

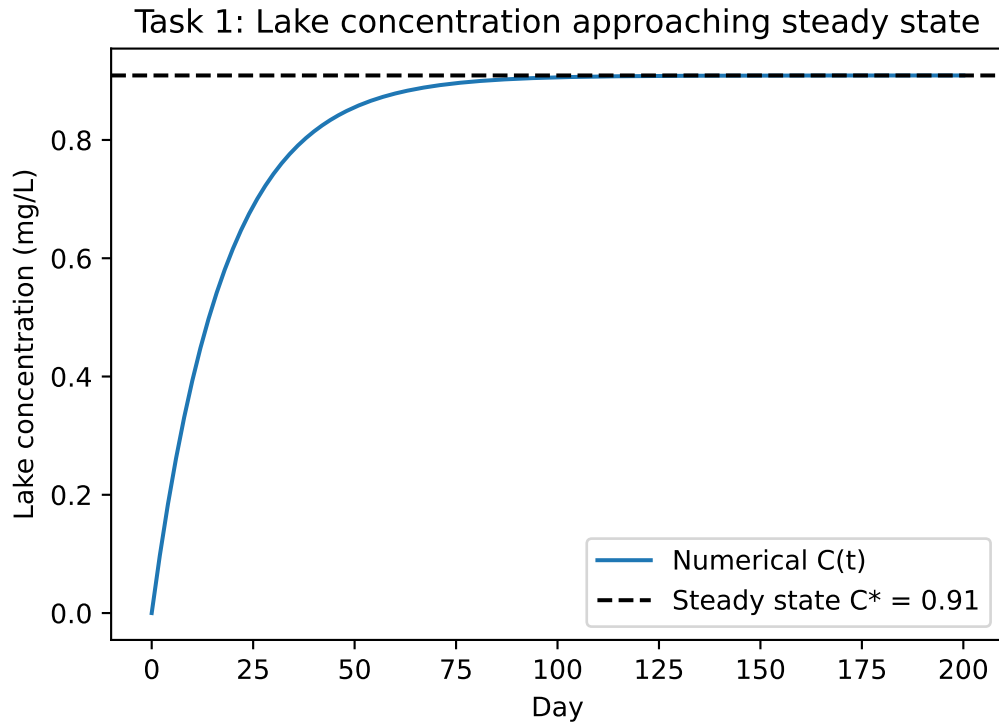
# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V  # the rate of change
    C = C + dCdt * dt                          # forward difference
    history.append(C)

# ---- Analytical steady state ----
C_star = (Q * C_in) / (Q + k * V)
print(f"Analytical steady state: {C_star:.2f} mg/L")
print(f"Numerical final value: {history[-1]:.2f} mg/L")
print(f"Difference: {abs(C_star - history[-1]):.4f} mg/L")

## Task 1
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(history, label="Numerical C(t)")
ax.axhline(C_star, color="k", linestyle="--", label=f"Steady state C* = {C_star:.2f}")
ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Task 1: Lake concentration approaching steady state")
ax.legend()
plt.show()

```

Analytical steady state: 0.91 mg/L
Numerical final value: 0.91 mg/L
Difference: 0.0000 mg/L



Answer to task B1: Analytical steady state: 0.91 mg/L. Numerical final value: 0.91 mg/L.
Difference: 0.0000 mg/L.

```
## Task 2: C=20
# ---- Parameters ----
V = 1.0e6      # lake volume (m3)
Q = 5.0e3      # river inflow (m3 per day)
C_in = 10.0    # incoming concentration (mg/L)
k = 0.05       # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0       # time step (days)
days = 200

# ---- Initial condition ----
C = 20.0       # polluted lake
```

```

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V # the rate of change
    C = C + dCdt * dt # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

# ---- Analytical steady state ----
C_star = (Q * C_in) / (Q + k * V)
print(f"Analytical steady state: {C_star:.2f} mg/L")
print(f"Numerical final value: {history[-1]:.2f} mg/L")
print(f"Difference: {abs(C_star - history[-1]):.4f} mg/L")

## Task 2 (same pattern, just plotting the C=20 history)
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(history, label="Numerical C(t), starting polluted")
ax.axhline(C_star, color="k", linestyle="--", label=f"Steady state C* = {C_star:.2f}")
ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Task 2: Polluted lake relaxing to steady state")
ax.legend()
plt.show()

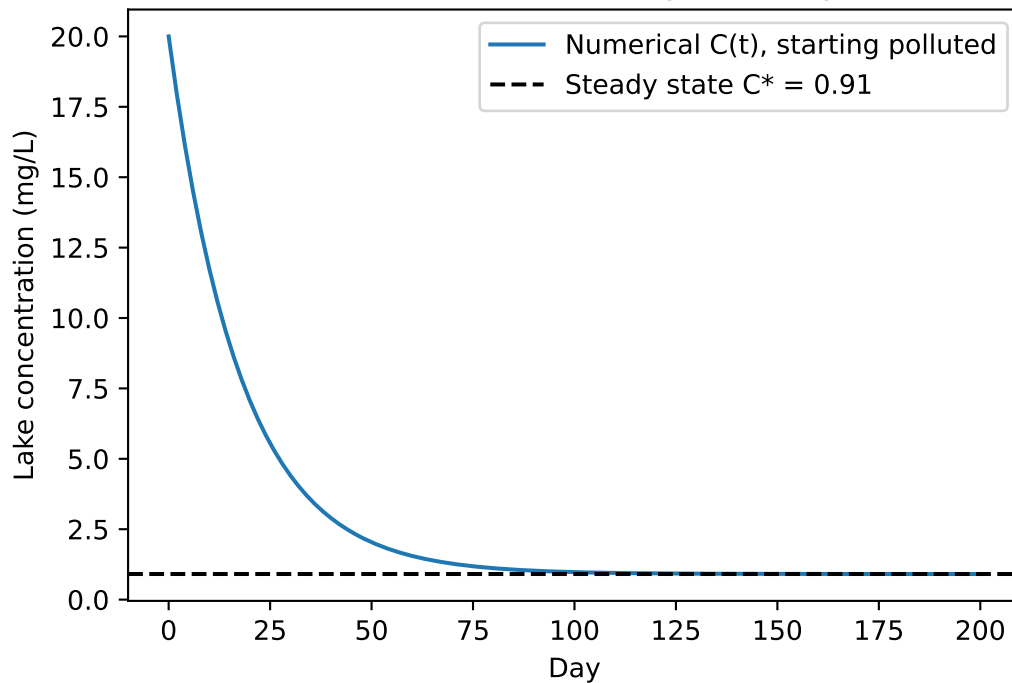
```

```

Started at 20.00 mg/L
After 10 days: 11.75 mg/L
After 50 days: 2.04 mg/L
After 200 days: 0.91 mg/L
Analytical steady state: 0.91 mg/L
Numerical final value: 0.91 mg/L
Difference: 0.0002 mg/L

```


Task 2: Polluted lake relaxing to steady state



Answer to task B2: The lake concentration decays exponentially from 20 mg/L down to the same steady state of 0.91 mg/L that the clean lake reached

```
## Task 3: Double inflow (Q=10,000 m3/day)
# ---- Parameters ----
V = 1.0e6          # lake volume (m3)
Q = 10.0e3         # river inflow (m3 per day)
C_in = 10.0        # incoming concentration (mg/L)
k = 0.05           # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0           # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0            # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
```

```

for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V    # the rate of change
    C = C + dCdt * dt                             # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

# ---- Analytical steady state ----
C_star = (Q * C_in) / (Q + k * V)
print(f"Analytical steady state: {C_star:.2f} mg/L")

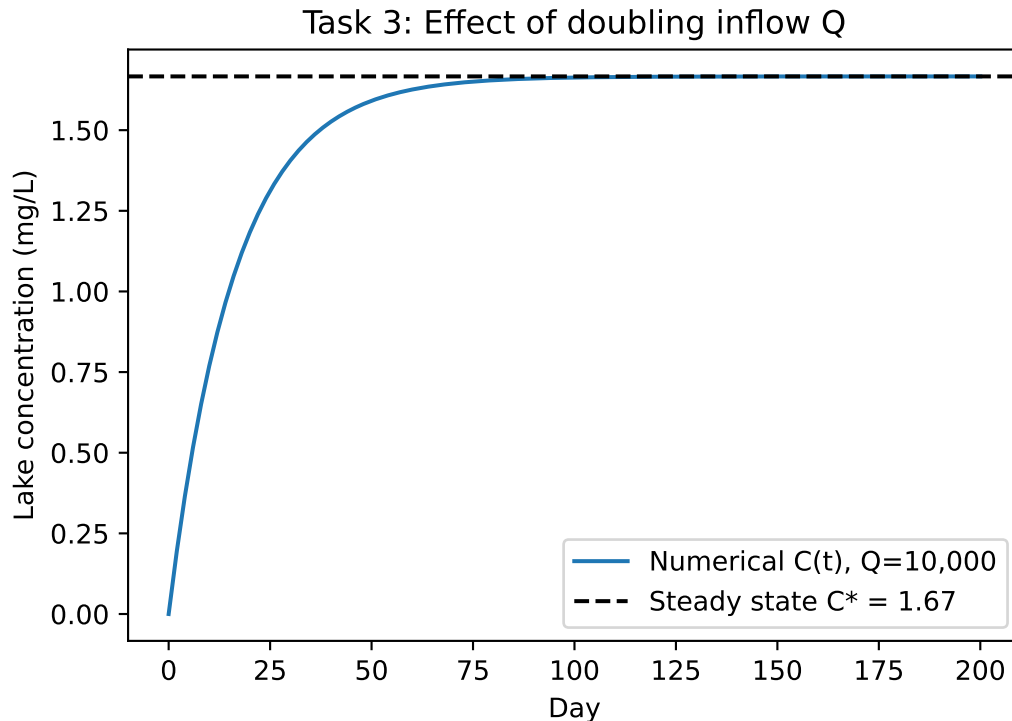
## Task 3 (same pattern, with Q doubled)
fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(history, label="Numerical C(t), Q=10,000")
ax.axhline(C_star, color="k", linestyle="--", label=f"Steady state C* = {C_star:.2f}")
ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Task 3: Effect of doubling inflow Q")
ax.legend()
plt.show()

```

```

Started at 0.00 mg/L
After 10 days: 0.77 mg/L
After 50 days: 1.59 mg/L
After 200 days: 1.67 mg/L
Analytical steady state: 1.67 mg/L

```



Answer to task B3: *The steady state rises.* Because $C^* = Q \cdot C_{in} / (Q + k \cdot V)$, when Q (the numerator) is increased proportionally compared to the denominator ($Q + k \cdot V$), the difference between C^* and C_{in} decreases.

```
## Task 4: sweeping parameters
# ---- Parameters ----
V = 1.0e6          # lake volume (m^3)
Q = 10.0e3         # river inflow (m^3 per day)
C_in = 10.0        # incoming concentration (mg/L)
k_test = [0.02, 0.05, 0.1, 0.2] # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0           # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0            # clean lake

# ---- Storage ----
history = [C]
```

```

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V # the rate of change
    C = C + dCdt * dt # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

# ---- Analytical steady state ----
C_star = (Q * C_in) / (Q + k * V)
print(f"Analytical steady state: {C_star:.2f} mg/L")

## Task 4: sweeping k - following the Chapter 2.2 pattern
fig, ax = plt.subplots(figsize=(7, 4))

for k_test in [0.02, 0.05, 0.1, 0.2]:
    C = 0.0
    history = [C]
    for _ in range(days):
        dCdt = (Q * C_in - Q * C - k_test * V * C) / V # use k_test, not k
        C = C + dCdt * dt
        history.append(C)
    ax.plot(history, label=f"k = {k_test}")

ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Task 4: Effect of decay rate on the reservoir")
ax.legend()
plt.show()

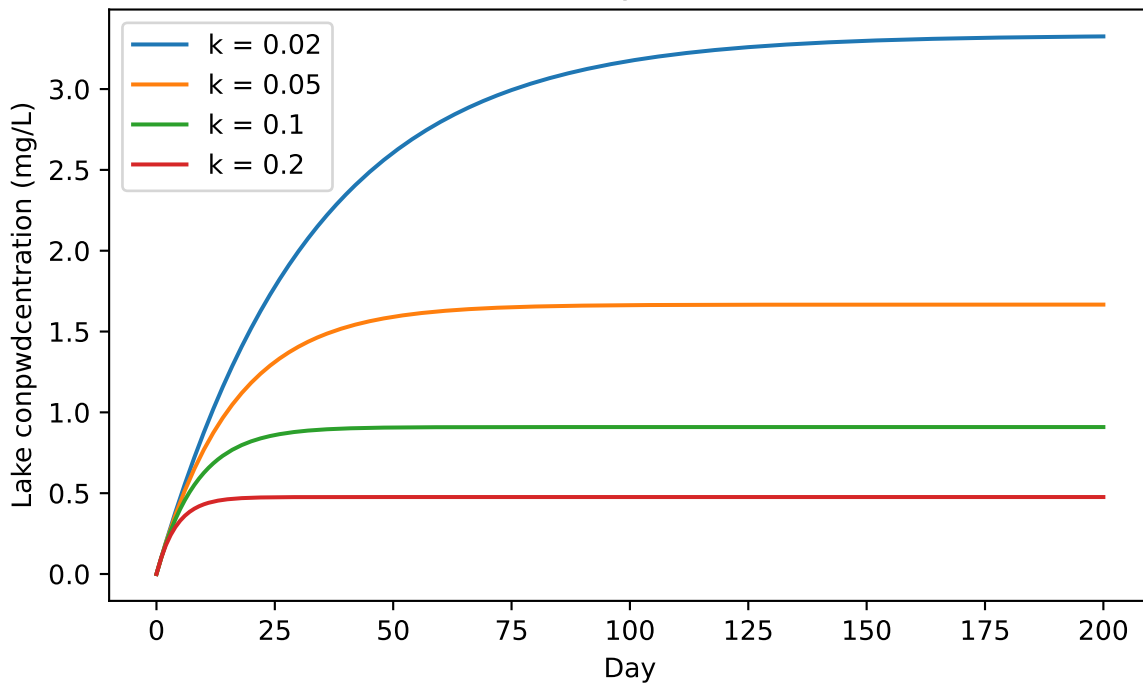
```

```

Started at 0.00 mg/L
After 10 days: 0.77 mg/L
After 50 days: 1.59 mg/L
After 200 days: 1.67 mg/L
Analytical steady state: 1.67 mg/L

```

Task 4: Effect of decay rate on the reservoir



Answer to task B4: Increasing k lowers the steady state and makes the lake concentration steady faster compared to lower values of k , because a bigger decay rate removes pollutants more aggressively each day.

Reflection

The coding process was relatively straightforward, but I was confused about why the model oscillated when $k \cdot dt > 1$, and thought that it was an issue with the code, but after looking at the equations, realised that it was the way the code handled the model/equation. One thing I would change in part B is graphing the first 3 tasks onto one graph because this would reduce the amount of code chunks I have, that are almost all identical (minus a few tweaks). This would make it easier to parse through the code in its entirety as well. The thing I got stuck on was initially rendering the lab assignment in Quarto, but after some trial and error, as well as resetting the envsci704 folder and .venv folder, I was able to render the lab assignment successfully. Next time, I would make sure to check and ensure that my environment is set up correctly before starting the lab assignment.

Checklist before you push

- ☐ Every task has a code cell **and** a short prose answer

- ☐ Every figure has axis labels and a title
- ☐ Your name and student ID are in the YAML header
- ☐ The file renders cleanly (no red error boxes in the HTML output)
- ☐ Filename is `lab-XX-topic.qmd`, not `Untitled.qmd`